STOCKWHISK SAAS PLATFORM

DOC-03

Role & Granular Permission Matrix

Enterprise Access Control, Role Hierarchies, and Security Scopes

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Security & Governance	Security Auditors, Admins, Developers	v1.4.0 Production	✅ Verified & Approved

Role and Permission Matrix

StockWhisk utilizes a Role-Based Access Control (RBAC) system to govern what users can see and do within the platform.

1. Roles Definition

1. **Platform Super Admin:** Platform owner/operator. (`is_staff=True` , `is_superuser=True`). Has unrestricted access to all platform configurations, billing, and all tenant data across the entire system.
2. **Shop Owner:** Primary account holder for a specific tenant. (`is_owner=True`). Has unrestricted access to their specific shop, including billing, subscription management, and user/role administration.
3. **Shop Manager:** A configurable role intended for senior staff. Can be customized by the Shop Owner, but typically has broad access to manage inventory, sales, reports, and basic settings.
4. **Cashier:** A frontline role. Typically restricted to creating sales via the POS, viewing basic product info, and perhaps processing returns. Restricted from sensitive financial reports or backend configuration.
5. **Reseller:** A platform partner. (`is_reseller=True`). Access is strictly limited to the Reseller portal to view their referrals and commissions; they do not have access to any specific shop's internal data.

2. Permission Matrix

The following table maps core permissions (from the `accounts.Permission` model) to standard roles.

(Note: Shop Owner has all permissions implicitly. Manager and Cashier permissions are highly configurable by the Shop Owner; the below represents a typical default configuration.)

PERMISSION KEY	DESCRIPTION	PLATFORM ADMIN	SHOP OWNER	SHOP MANAGER (TYPICAL)	CASHIER (TYPICAL)	RESELLER
`view_dashboard`	View main analytics dashboard	✅	✅	✅	❌	❌
`view_products`	View product catalog	✅	✅	✅	✅	❌
`manage_products`	Create/Edit/Delete products	✅	✅	✅	❌	❌
`view_inventory`	View stock levels	✅	✅	✅	✅	❌
`manage_inventory`	Adjust stock, transfers	✅	✅	✅	❌	❌
`create_sale`	Access POS, generate invoices	✅	✅	✅	✅	❌
`view_sales`	View past invoices	✅	✅	✅	✅	❌
`edit_sales`	Modify existing invoices	✅	✅	❌	❌	❌
`process_return`	Handle sales returns	✅	✅	✅	✅	❌
`view_customers`	View customer CRM profiles	✅	✅	✅	✅	❌
`manage_customers`	Edit/Delete customers, debts	✅	✅	✅	❌	❌
`manage_purchasing`	Handle POs and Suppliers	✅	✅	✅	❌	❌
`view_reports`	Access financial/sales reports	✅	✅	✅	❌	❌
`manage_expenses`	Record operational expenses	✅	✅	✅	❌	❌
`view_service`	View warranty/service tickets	✅	✅	✅	✅	❌
`manage_service`	Edit/Update service tickets	✅	✅	✅	❌	❌
`manage_users`	Add staff, change roles	✅	✅	❌	❌	❌
`manage_settings`	Edit shop profile, VAT, configs	✅	✅	❌	❌	❌

PERMISSION KEY	DESCRIPTION	PLATFORM ADMIN	SHOP OWNER	SHOP MANAGER (TYPICAL)	CASHIER (TYPICAL)	RESELLER
`view_accounting`	View ledgers, P&L	✅	✅	✅	❌	❌
`manage_accounting`	Alter ledgers, investments	✅	✅	❌	❌	❌

3. Implementation Details

3.1 Backend Enforcement (Django DRF)

Permissions are enforced at the API level using custom Django REST Framework permission classes (e.g., `HasPermission('create_sale')`).
- The middleware attaches the authenticated `User` to the request.
- The permission class checks if the user's assigned `Role` contains the required `Permission` for the current tenant context.
- Owners (`is_owner=True`) automatically bypass these checks for their own shop.

3.2 Frontend Enforcement (Next.js)

The frontend uses a `can(permission_key)` helper function provided by an authentication context or hook.
- UI elements (buttons, menu links) are conditionally rendered based on the result of `can()`.
- Route guards prevent unauthorized access to specific pages, redirecting users if they lack required permissions.

3.3 Role Management

- Roles are created and managed by the Shop Owner within the "Settings > Roles" area.
- The Owner defines a Role (e.g., "Senior Cashier") and toggles specific permissions for that role.
- Users (staff) are then invited to the shop and assigned one of these custom roles.

3.4 Demo Mode Restrictions

The system includes a `DemoReadOnlyMiddleware`. If a shop is flagged as a "Demo Shop", this middleware automatically intercepts and blocks unsafe HTTP methods (`POST`, `PUT`, `PATCH`, `DELETE`), returning a 403 Forbidden. This allows users to safely explore the platform without altering demo data, regardless of their assigned role permissions within that demo shop.

3.5 Impersonation

Platform Super Admins have the ability to "impersonate" a shop. This allows them to log in and view the application exactly as the Shop Owner would, aiding in customer support and troubleshooting. This action is logged securely in the system audit trails.